



Elixirr



웹 데이터 수집 1

크롤링/스크래핑

본 자료의 일부 또는 전체는 Elixirr의 사전 서면 승인 없이 외부로 배포, 인용 또는 복제하는 행위를 금합니다.

주요 라이브러리



BeautifulSoup



라이브러리 설치

```
pip install requests
```

```
pip install beautifulsoup4
```

```
pip install selenium
```

```
pip install openpyxl
```

```
pip install lxml
```

```
pip install pillow
```

```
pip install konlpy
```

```
pip install wordcloud
```

웹 크롤링 vs 웹 스크래핑

웹크롤링(Web Crawling)	웹 스크래핑(Web Scraping)
✓ 웹 페이지를 탐색하고 색인화 - 자동화된 방식으로 웹페이지들을 순회하면서 링크를 따라 다니는 것 - 검색엔진의 봇이 대표적인 예시	✓ 웹 페이지에서 구조화된 데이터 추출

※ 위 두 기술이 함께 사용되어 웹 데이터를 수집.

※ 본 자료는 교육생 본인 학습용으로만 사용 가능합니다. 외부에 공개하지 말아주세요.

Copyright © 2025 Jayoung Kim All rights reserved

활용 예시

가격 비교

여러 온라인 쇼핑몰의 제품 가격 수집
|| 최저가 탐색

뉴스 모니터링

특정 주제/이슈에 대한 기사 수집
|| 평판관리, 정책수립, 선거 캠페인 등

주식 정보 수집

가격, 거래량, 경제지표 등 데이터 수집
|| 시장동향 파악, 투자 의사결정

소셜미디어 분석

소셜미디어 게시물 수집
|| 소비자 트렌드 파악, 홍보전략 수립

고객 리뷰 분석

제품/서비스에 대한 고객 리뷰 수집
|| 제품 개선, 고객 만족도 상승

채용 정보 수집

다양한 취업 사이트의 채용공고 수집
|| 맞춤 채용 정보 조회

웹데이터 수집 주의사항

법적 주의사항

- ✓ 웹사이트 콘텐츠는 대부분 **저작권**으로 보호됨
- ✓ 상당한 투자와 노력으로 구축된 데이터베이스는 제작자의 권리가 인정됨
- ✓ 무단 크롤링은 저작권법과 부정경쟁방지법 위반이 될 수 있음

기술적 주의사항

- ✓ 과도한 요청으로 **서버에 부담**을 주지 않도록 크롤링 간격 조정
과도한 요청 시 영업방해로 인정될 수 있음.
- ✓ robots.txt 파일의 크롤링 규칙 확인 및 준수
- ✓ 기술적 제한을 우회하는 행위 금지

윤리적 고려사항

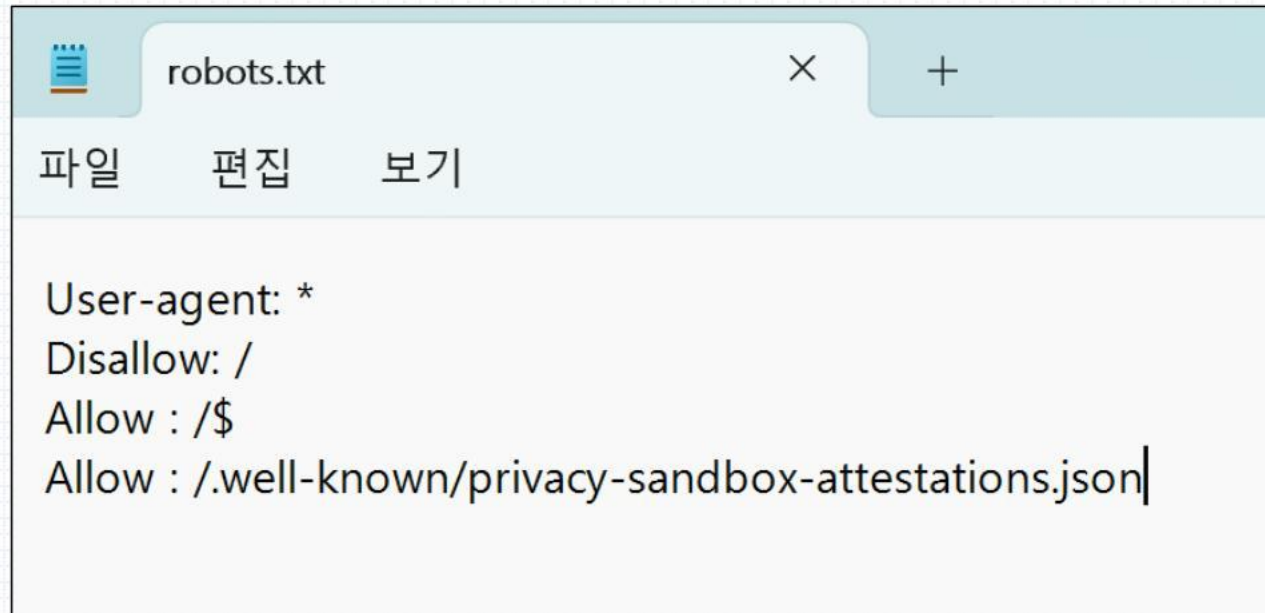
- ✓ 상업적 목적으로 사용 시 저작권자의 허가 필요
- ✓ 경쟁 업체의 데이터를 무단으로 활용하지 않도록 주의

웹데이터 수집 주의사항

▪ robot.txt 확인 방법

웹브라우저 주소창에 해당 웹사이트 주소 뒤에 /robots.txt를 입력하여 robots.txt 파일 다운로드

(예) `http://www.naver.com/robots.txt`



- **User-agent** : 규칙을 적용할 크롤러
- **Disallow** : 접근 차단할 경로
- **Allow** : 접근 허용할 경로
- **Crawl-delay** : 크롤링 요청 간 대기 시간(초)

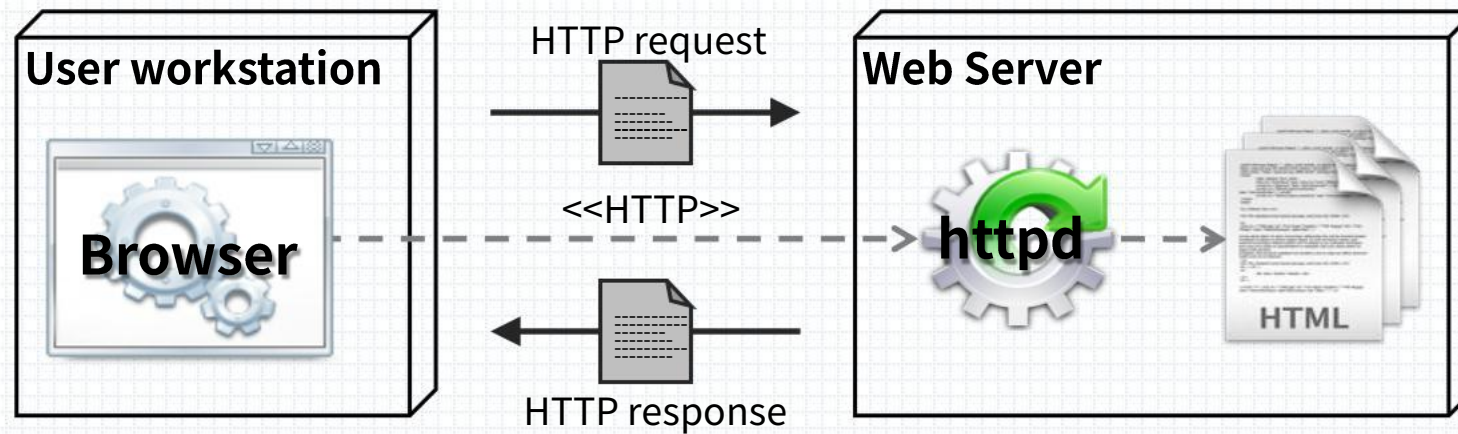
※robot.txt는 권고 사항이며 강제성은 없음
이 규칙을 준수하는 것은 웹 크롤링의 에티켓이나,
법정 분쟁 발생 시에는 고려 대상이 될 수 있음

웹페이지의 동작 원리

웹페이지의 동작 원리

▪ HTTP Request와 HTTP Response

- 브라우저가 서버의 페이지를 요청하면 서버는 해당 파일을 찾은 후 HTTP 응답을 통해 전송
- 브라우저는 응답된 페이지를 해석하여 화면에 보여줌



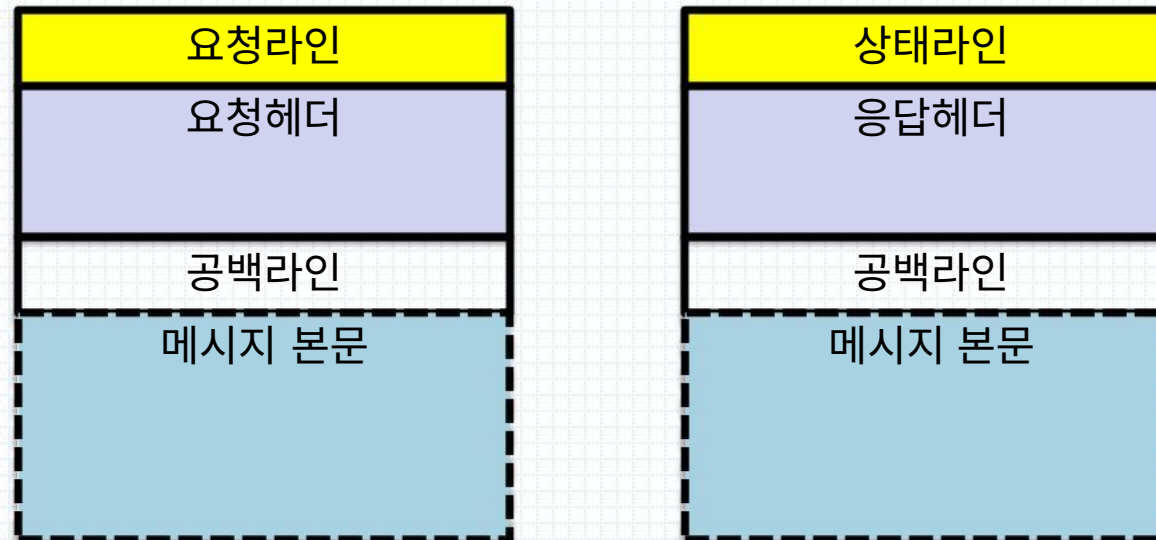
- ✓ request : 사용자가 원하는 파일 또는 리소스의 위치와 브라우저에 관한 정보를 포함
- ✓ response : 응답객체. 텍스트, 그래픽 등의 정보를 포함할 수 있음

웹페이지의 동작 원리

▪ HTTP Message

- HTTP Message는 크게 요청과 응답 메시지로 구분
- HTTP Message는 라인, 헤더, 바디 영역으로 나뉘며, 헤더와 바디 영역은 공백으로 구분
- 바디 영역을 제외한 나머지 영역(요청, 헤더)의 각 라인은 연속된 CR/LF(\r\n) 문자로 구분

HTTP Request 메시지의 구성 HTTP Response 메시지의 구성



웹페이지의 동작 원리

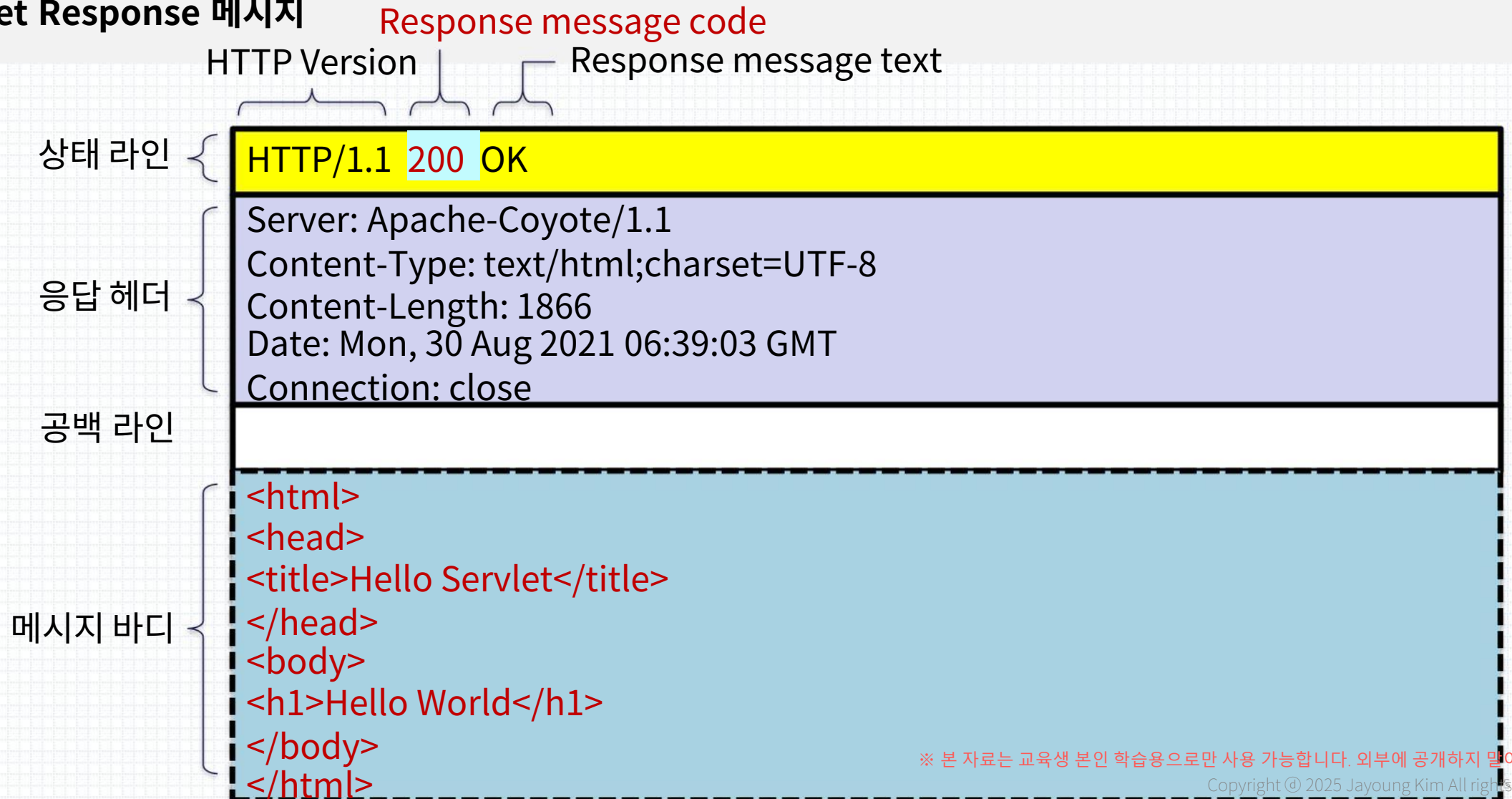
▪ HTTP Get Request 메시지



※ 맥 값일 줄을 ※ 본 자료는 교육생 본인 학습용으로만 사용 가능합니다. 외부에 공개하지 말아주세요.

웹페이지의 동작 원리

▪ HTTP Get Response 메시지

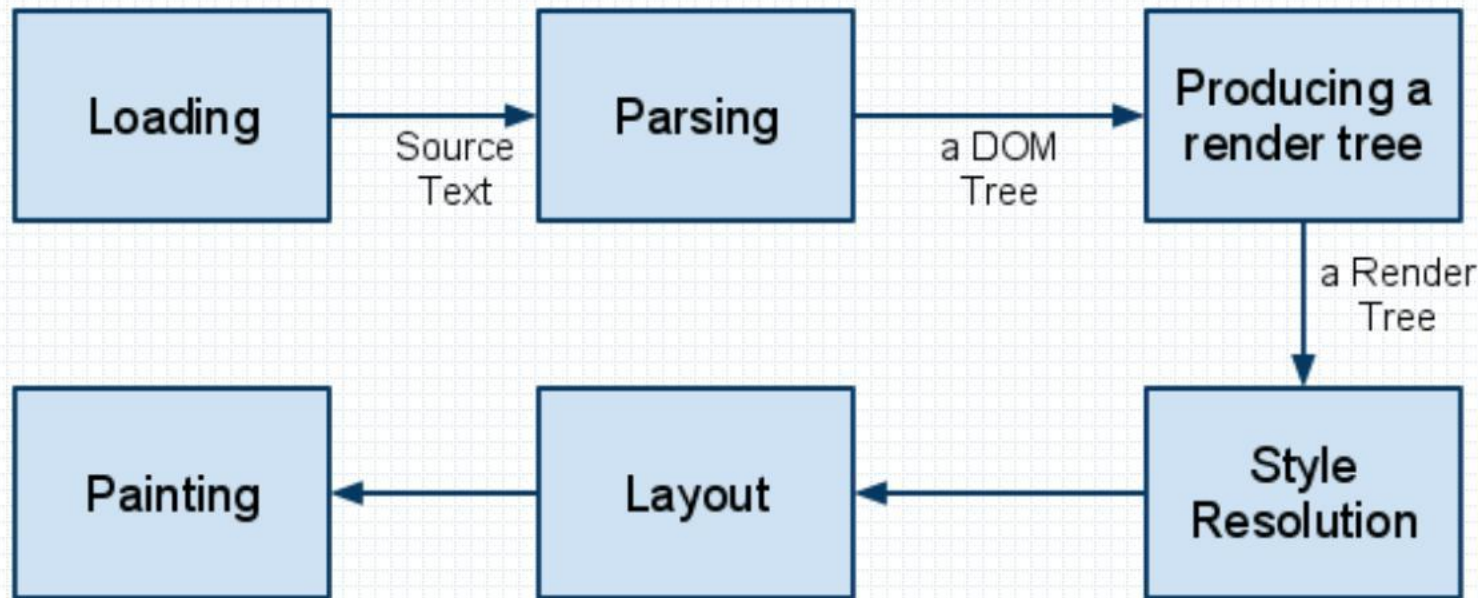


웹페이지의 동작 원리

▪ 브라우저가 하는 일

브라우저의 주요 컴포넌트는

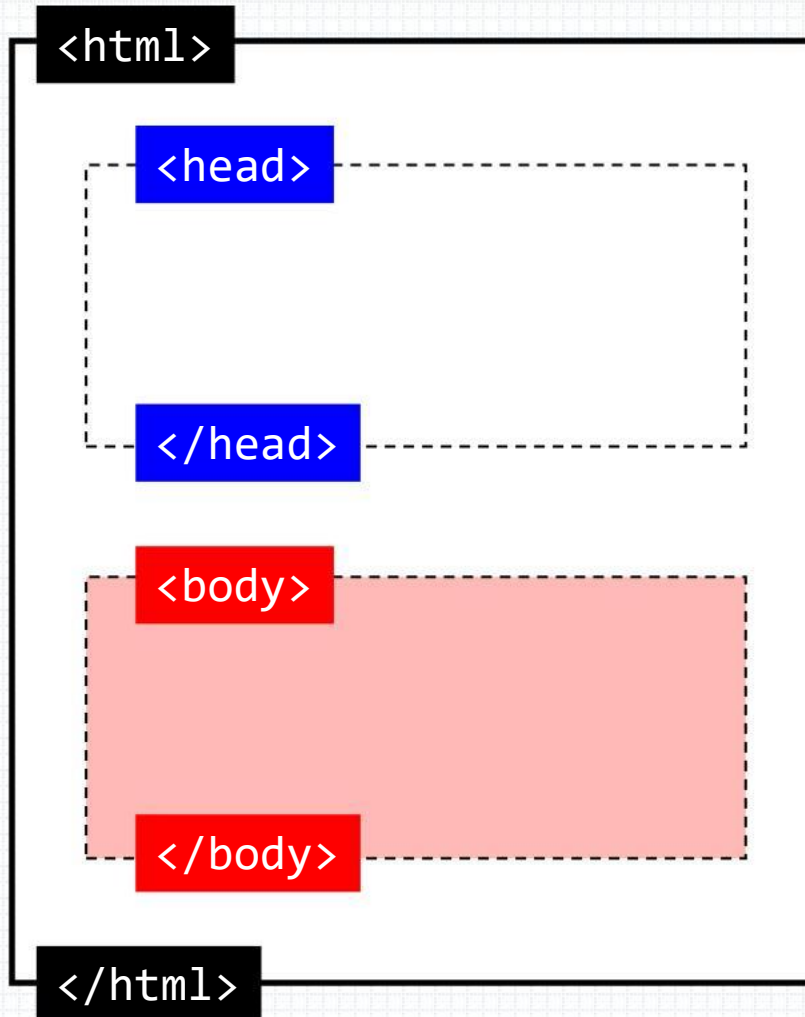
`렌더링 엔진(레이아웃 엔진)`과 `자바스크립트 해석기`입니다.



- 불러오기(Loading)
- 파싱(Parsing)
- 자바스크립트 실행
- 레이아웃(Layout)작업
- CSS 처리
- 그리기
- 이벤트 처리

HTML 문서의 기본 구조

HTML 문서의 기본 구조



`<html>~</html>`

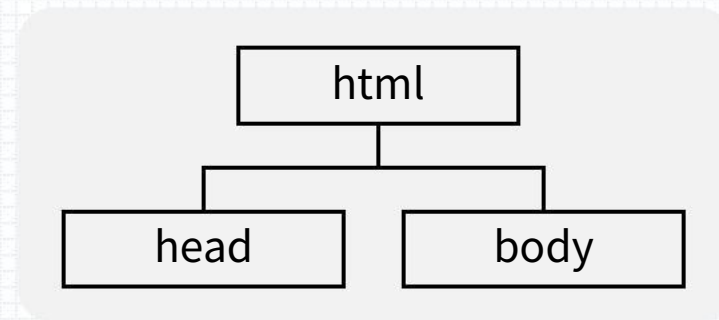
웹문서의 시작과 끝을 나타내는 태그

`<head>~</head>`

웹브라우저가 웹문서를 해석하는 데 필요한 정보

`<body>~</body>`

웹브라우저 화면에 나타나는 내용



HTML 문서의 구성요소

태그 (tag)	- html 요소를 정의하는 데 사용되는 코드. 시작태그, 종료태그가 있음. (예) <code><p>hello</p></code>, <code></code>, <code>
</code>
요소 (element)	- 태그와 태그 사이에 있는 내용을 합쳐서 '요소(element)'라고 부른다. - 내용을 갖는 요소, 내용을 갖지 않는 요소가 있음 (예) <code><p>hello</p></code>, <code></code>, <code>
</code>
속성 (attribute)	- 요소에 추가적인 정보를 제공하는 부분. - 열린 태그에서 정의. (속성이름=속성값) (예) <code></code>
내용 (content)	- 시작태그와 종료태그 사이에 위치한 내용. 내용을 갖는 태그, 갖지 않는 태그가 있음. (예) <code><p>hello</p></code>

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
```

```
<html lang="en">
```

시작태그

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Document</title>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

종료태그

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head> 시작태그
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head> 종료태그
<body>

</body>
</html>
```

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

종료태그가 없는 태그도 있음

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```


HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- ✓ 태그와 태그 사이에 있는 내용을 합쳐서 ‘요소(element)’라고 부른다.
- ✓ 내용을 갖는 요소, 갖지 않는 요소가 있다.

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- ✓ 태그와 태그 사이에 있는 내용을 합쳐서 ‘요소(element)’라고 부른다.
- ✓ 내용을 갖는 요소, 갖지 않는 요소가 있다.

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- ✓ 태그와 태그 사이에 있는 내용을 합쳐서 ‘요소(element)’라고 부른다.
- ✓ 내용을 갖는 요소, 갖지 않는 요소가 있다.

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- ✓ 태그와 태그 사이에 있는 내용을 합쳐서 ‘요소(element)’라고 부른다.
- ✓ 내용을 갖는 요소, 갖지 않는 요소가 있다.

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- ✓ 태그와 태그 사이에 있는 내용을 합쳐서 ‘요소(element)’라고 부른다.
- ✓ 내용을 갖는 요소, 갖지 않는 요소가 있다.

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- ✓ 태그와 태그 사이에 있는 내용을 합쳐서 ‘요소(element)’라고 부른다.
- ✓ 내용을 갖는 요소, 갖지 않는 요소가 있다.

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- ✓ 요소에 추가적인 정보를 제공하는 부분이다.
- ✓ 열린 태그에서 정의한다.
- ✓ 하나의 tag에는 여러 개의 attribute를 사용할 있다.
- ✓ 여러 개의 attribute가 있는 경우 공백으로 구분한다.

※ 본 자료는 교육생 본인 학습용으로만 사용 가능합니다. 외부에 공개하지 말아주세요.

HTML의 구성요소

tag

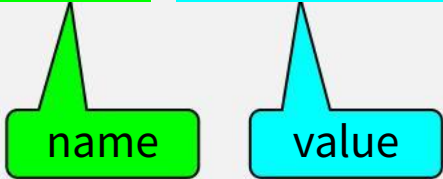
element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```



|| attribute는 **name**과 **value**를 가진다.

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- ✓ 시작태그와 종료태그 사이에 위치한 내용.
- ✓ 내용을 갖는 태그, 갖지 않는 태그가 있음.

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

HTML의 구성요소

tag

element

attribute

content

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

CSS 선택자

CSS 선택자

- CSS
 - ✓ 웹문서의 콘텐츠, 레이아웃, 글꼴 및 시각적 요소를 담당
- CSS 선택자
 - ✓ CSS를 이용해서 HTML 문서에 시각적 요소를 부여할 때 문서 내의 어느 요소에 부여할지 결정하기 위해 사용되는 것
 - ✓ 웹스크래핑에서는 웹문서에서 특정 요소를 선택하여 추출할 때 사용

CSS 선택자의 종류

https://developer.mozilla.org/ko/docs/Web/CSS/CSS_selectors

■ 기본 선택자

선택자 종류	스타일 적용 범위	사용법
전체 선택자 Universal Selector	문서의 모든 요소	*
tag 선택자 Type Selector	특정 태그를 사용한 모든 요소	태그명
class 선택자 Class Selector	동일한 class명을 사용하는 모든 요소	.class명
id 선택자 ID Selector	해당 id를 가진 단일 요소 ID는 페이지에서 유일해야 함	#id명
그룹 선택자 Group Selector	쉼표로 구분된 모든 선택자에 해당하는 요소	선택자1, ..., 선택자n
속성 선택자 Attribute Selector	지정된 속성을 가진 모든 요소	[attribute]
	지정된 속성 값을 가진 모든 요소	[attribute="value"]

CSS 선택자의 종류

- 속성 선택자

속성선택자	설명
태그[속성]	특정 속성이 있는 요소를 선택
태그[속성=속성값]	특정 속성값이 있는 요소를 선택
태그[속성~=값]	여러 값 중 특정 속성값이 포함된 속성 요소를 선택
태그[속성 =값]	특정 속성값이 포함된 속성 요소를 선택 하이픈(-) 또는 공백으로 구분된 값의 첫 부분을 대상으로 한다.
태그[속성^=값]	특정 속성값으로 시작하는 속성 요소를 선택
태그[속성\$=값]	특정 속성값으로 끝나는 속성 요소를 선택
태그[속성*=값]	일부 속성값이 일치하는 요소를 선택

CSS 선택자의 종류

■ 고급 선택자

선택자 종류	스타일 적용 범위	사용법
자식 선택자 Child Selector	특정 부모 요소의 직계 자식 요소를 선택	parent > child
후손 선택자 Descendant Selector	특정 조상 요소의 모든 후손 요소를 선택	ancestor descendant
인접 형제 선택자 Adjacent Sibling Selector	특정 요소 바로 다음에 오는 형제 요소를 선택	A + B
일반 형제 선택자 General Sibling Selector	특정 요소 뒤에 위치한 모든 형제 요소를 선택	A ~ B
가상 클래스 선택자 Pseudo-classes	특정 상태에 있는 요소를 선택	:pseudo-class (예) a:hover {} // 마우스포인터로 요소를 가리키고 있을 때만 요소를 선택
가상 요소 선택자 Pseudo-elements	요소의 특정 부분에 스타일을 적용	::pseudo-element (예) p::first-line // 요소 내부의 첫번째 줄 선택
복합 선택자 Combinator Selectors	해당 요소 및 클래스를 모두 가진 요소 선택	element.class .class.class .class.class.class 등
부정 선택자 Negation Pseudo-class	지정된 선택자와 일치하지 않는 요소를 선택	:not(selector)

CSS 선택자(CSS selector)

- n번째 태그만 선택

특정 유형(type)의 형제요소 중 특정 순서에 있는 요소 선택

선택자: nth-of-type(**n**)

p:nth-of-type(3)	# 3번째 p요소 선택
p:nth-of-type(even)	# 짝수번째 p요소 선택
p:nth-of-type(odd)	# 홀수번째 p요소 선택
p:nth-of-type(3n)	# 3의 배수번째 p 요소 선택

CSS 선택자(CSS selector)

- 부정 선택자

괄호 안에 지정된 선택자와 일치하지 않은 요소 선택

찾을선택자:not(**제외할선택자**)

`a:not(.example)` # a요소 중 example 클래스 제외

`div:not(.example, #myid)` # div요소 중 example클래스, myid 아이디 제외

웹페이지의 유형에 따른 크롤링 유형

정적 페이지 크롤링

- 한 번에 모든 HTML 문서를 응답 받는 페이지
- 고정된 페이지를 크롤링하는 페이지

사용 라이브러리

- **requests**
 - http 요청을 보내고 응답 받음
 - 웹브라우저 열지 않음
- **BeautifulSoup**
 - html문서 파싱, 데이터 추출



동적 페이지 크롤링

- HTML을 응답 받은 후 일부 내용을 JavaScript가 동적으로 생성하는 페이지
- 사용자와 상호작용하며 콘텐츠가 동적으로 변하는 페이지

사용 라이브러리

- **selenium**
 - 웹브라우저를 열어 웹페이지 로드
 - 웹요소로부터 데이터 추출

